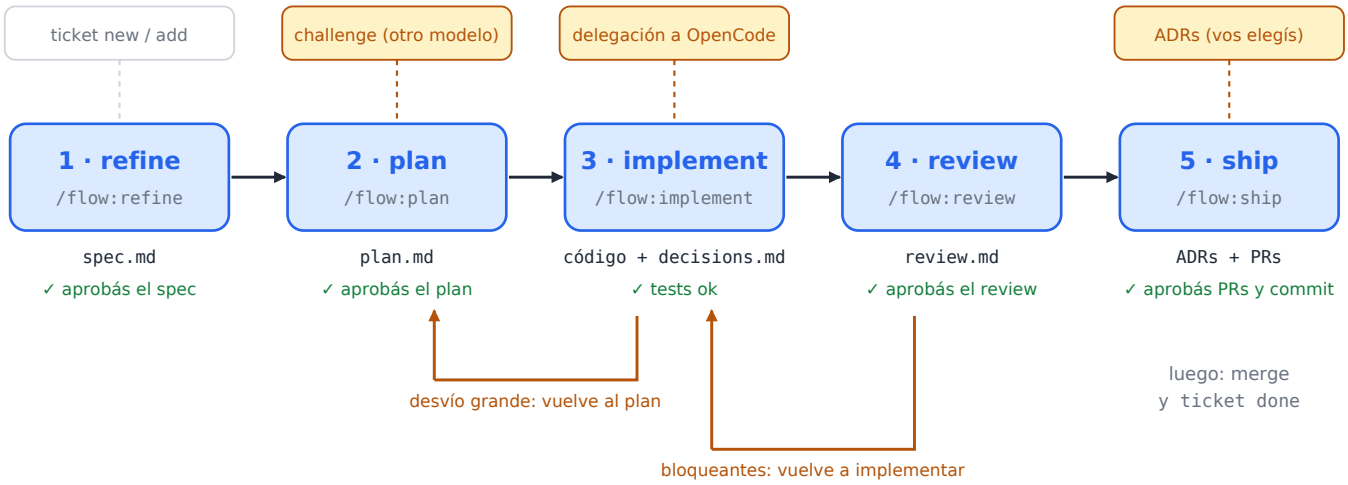
Flujo de tickets con agentes

Guía de **agent-workflow**: plugin **flow** de Claude Code + comando **ticket**. Cada fase deja un archivo en la carpeta del ticket y la siguiente lo lee: sin copy-paste entre chats, y se retoma en una sesión nueva sin perder contexto.



1. Instalar

```
git clone https://github.com/<usuario>/agent-workflow ~/projects/agent-workflow
cd ~/projects/agent-workflow && bash install.sh
```

Requisitos: Claude Code, `git` y `gh` (con `gh auth login`). Opcionales: `opencode` con OpenCode Go (delegación, challenger, reviewer) y Obsidian. Clonalo fuera de Desktop/Documents: macOS bloquea esas carpetas a algunas apps.

install.sh (se puede repetir): enlaza `ticket` en `~/.local/bin`, registra la carpeta como marketplace local e instala el plugin, enlaza los agentes de OpenCode y crea el vault de Obsidian. Después, en Claude Code: `/reload-plugins`. Para actualizar: `git pull` y `/reload-plugins`.

2. Configurar cada proyecto

```
ticket init mi-repo     # crea mi-repo/.agent-workflow.md
```

Por repo: plataforma de tareas, branch, formato de commit, si `/flow:implement` stagea o commitea, PRs, tests, delegación, challenge, reviewer, carpeta de ADRs y datos sensibles. Es una config **personal y local**: queda en `.git/info/exclude` y nunca se commitea; `ticket add` la enlaza por symlink en cada worktree. Sin ella se usan defaults y no se delega.

3. Usar el flujo

```
ticket new mi-repo#12                          # desde un issue de GitHub
ticket new CAN-123 export-pdf --link <url-tarea>   # tarea de otra plataforma
cd "$(ticket path 12)" && claude
```

Paso	Dónde	Qué hace	Checkpoint
`/flow:refine`	agente	Investiga y escribe `spec.md`: ACs, alcance, preguntas para producto	Aprobás el spec
`ticket add repo`	terminal	Suma repos: un worktree por repo, misma branch	
`/flow:plan`	agente	Escribe `plan.md`: decisiones, contratos, tareas. Corre el challenge	Aprobás el plan
`/flow:implement [repo]`	agente	Ejecuta tareas, deja en staging, registra desvíos en `decisions.md`	Tareas tildadas, tests ok
`/flow:review`	agente	Resumen de decisiones + PR reviewer (opcional) + UI con Playwright → `review.md`	Aprobás el review

Paso	Dónde	Qué hace	Checkpoint
`/flow:ship`	agente	Propone ADRs, redacta PRs y commit; el `shipper` pushea y abre los PRs	Aprobás PRs y commit
`/flow:status`	agente	Dónde quedó el ticket y qué sigue (para retomar)	
`ticket ls / done`	terminal	Ver tickets activos / cerrar y archivar tras el merge	

Entre fases conviene abrir una **sesión nueva**: el contexto vive en los archivos.

4. Las fases en detalle

Fase	Qué hace	Cuándo termina
`refine`	Lee el ticket y los ADRs previos, explora los repos en solo lectura. Escribe problema, **fuera de alcance**, criterios de aceptación `AC-1…` (Dado/Cuando/Entonces), supuestos, preguntas para producto con default propuesto y riesgos. No toca código.	Spec aprobado → `fase: plan`
`plan`	Decisiones de arquitectura con alternativas, contratos entre repos, orden de ejecución, tareas `T1…` chicas (un commit cada una) con su AC y cómo se verifican, plan de testing y riesgos. Marca `[delegable]` lo que un modelo barato puede hacer. Todo AC tiene una tarea.	Plan aprobado (con el challenge respondido)
`implement`	Una tarea a la vez, solo dentro de los worktrees: implementa, corre tests y lint, deja en staging (o commitea por tarea). Nunca pushea. Desvío chico → `D-n` en `decisions.md`; desvío grande (contrato, AC, alcance, datos, infra) → **frena y pregunta**.	Todas las tareas tildadas y tests ok
`review`	Revisores que no vieron cómo se escribió el código: `decisions-summarizer`, PR reviewer de OpenCode y `ui-verifier` (Playwright). Consolida veredicto, bloqueantes, decisiones para validar y cobertura de ACs. No arregla nada sin que se lo pidas.	Review aprobado
`ship`	Propone ADRs (elegís cuáles), redacta un PR por repo, propone el commit y espera tu OK. Luego el `shipper` (Haiku) pushea y abre los PRs, enlazándolos entre sí si son varios repos.	PRs abiertos → `ticket done` tras el merge

5. La carpeta de un ticket

```
~/tickets/mi-repo-12-login-google/
├── .ticket          metadatos: ID, branch, repos, origen de la tarea
├── status.md        fase (frontmatter), checkpoints, pendientes, log
├── spec.md          fase 1
├── plan.md          fase 2 (incluye la sección Challenge)
├── challenge.md     objeciones del challenger, tal cual
├── decisions.md     desvíos y decisiones durante la implementación
├── review.md        fase 4
├── pr-mi-repo.md    fase 5: título y cuerpo del PR aprobado
├── ui-review.md · ui-evidence/   verificación de UI
└── .wt/mi-repo/     worktree en feat/12-login-google (oculto p/ Obsidian)
```

6. Agentes y subagentes

Agente	Modelo	Qué hace
`decisions-summarizer`	Sonnet	Solo lectura. Compara el diff con spec, plan y decisiones
`ui-verifier`	Sonnet	Prueba la UI con Playwright en localhost/staging, datos sintéticos
`challenger`	Opus	Alternativa de Claude a `flow-challenger` (motor `claude`)
`shipper`	Haiku	Push y PRs con `gh`. Nunca force push, merge ni commits
`flow-coder` (OpenCode)	configurable	Implementa tareas `[delegable]`. No commitea, no pushea, no lee `.env`
`flow-reviewer` (OpenCode)	configurable	Code review de solo lectura
`flow-challenger` (OpenCode)	configurable	Cuestiona el plan, solo lectura

El modelo de cada agente de OpenCode se define por proyecto en `.agent-workflow.md` y se pasa con `-m`: se cambia sin tocar los agentes.

7. Delegación a OpenCode

Con `Delegar tareas: sí`, `/flow:implement` le pasa las tareas `[delegable]` a `flow-coder` con un brief. Claude **siempre** verifica: revisa el diff completo, corre tests y lint él mismo y recién ahí stagea. Si queda mal, la corrige o la rehace una vez. Cada delegación queda en el Log de `status.md` (ok / corregida / rehecha) para comparar modelos. `--no-delegate` hace todo con Claude. Para código de empresas o clientes, solo modelos con retención cero y solo si la empresa lo permite.

8. Challenge del plan

Al terminar `/flow:plan`, un challenger con contexto fresco revisa el plan: hasta **5 objeciones**, cada una con evidencia (archivo:línea, un AC o una sección del plan) y una alternativa concreta. Claude responde cada una en la sección **Challenge** de `plan.md` (aceptada o rechazada, con motivo) y recién ahí te pide aprobar. Si una objeción cambia el alcance o un AC queda "para decidir" y te la levanta. Motor: `opencode` (otro modelo) o `claude` (Opus). Se saltea en planes chicos o con `--skip-challenge`.

9. Memoria del proyecto: ADRs

Las decisiones que sobreviven al ticket van al repo como ADR en `docs/decisions/NNNN-titulo.md` (índice en `README.md`). `/flow:ship` las propone y vos elegís; entran en el mismo PR. `/flow:refine` y `/flow:plan` las leen al arrancar como restricciones; para ir contra una hay que decirlo explícitamente ("reemplaza ADR NNNN porque..."). El review avisa si el código contradice una.

10. Obsidian (opcional)

Abrí `~/tickets` como vault: `Tickets.base` muestra el tablero por fase y los archivados. El estado vive en el frontmatter de `status.md`. Los worktrees están en `.wt/` (oculta) para que no se indexe código. Es una capa de lectura, no una base de datos. Buscar en todo: `rg "texto" ~/tickets`.

11. Reglas de oro

- Sesión nueva entre fases: el contexto vive en los archivos.
- Desvío grande en la implementación → el agente frena y pregunta.
- Nada se commitea ni se pushea sin que apruebes en `/flow:ship`.
- Review de UI: dev server levantado y su URL en `status.md`.
- Nunca `.env`, credenciales ni datos reales de usuarios en el contexto de un agente.

12. Si algo falla

Síntoma	Qué hacer
"no es un repo git" / "no tiene commits"	`git init` y un commit inicial
No puede leer el issue o abrir el PR	`gh auth login` y que el repo tenga remoto en GitHub
Tarea de otra plataforma	Su MCP conectado y "Cómo leer una tarea" en la config; si no, pegá el texto
No corre delegación, challenger o reviewer	Revisá la config, `opencode agent list` y `opencode models`
No aparecen los `/flow:*`	`/reload-plugins` o `claude plugin list`
`ticket: command not found`	Agregá `~/.local/bin` al PATH y abrí otra terminal
`ticket done` se queja	Commiteá y `git push -u origin HEAD` en el repo indicado
Cambié una skill y no se refleja	`/reload-plugins`; si sigue igual, subí la versión del plugin y `claude plugin update flow`

Para cambiar cómo trabaja una fase, editá su `SKILL.md` en `plugin/skills/<fase>/` (es markdown). Las plantillas están en `templates/`.